

Must Explain — Assessment 1 Answers

1. Prompt engineering strategy for the 90–150 word constraint

The problem: Asking an LLM to "write between 90 and 150 words" in a single prompt is unreliable. The model may produce 60 words or 200 words and consider the constraint met.

What was built: A retry loop inside the n8n Code node (JavaScript, Groq Llama 3.3 70B via the OpenAI-compatible Chat Completions API).

```
Attempt 1 → generate copy → count words
  If 90–150: done
  If out of range: append corrective turn and retry (up to 5 times)
```

The corrective turn tells the model its exact word count and asks for a full rewrite — not a trim:

"Your previous draft was 74 words. Rewrite the full post to be between 90 and 150 words. Do not just trim — rewrite naturally so the post reads well and stays on topic."

Why a rewrite instruction rather than truncation: Truncating at 150 words leaves mid-sentence or mid-idea text. The model rewrites the entire post, so the result remains coherent and on-brand. The word count is checked with a JavaScript `split` (`copy.split(/\s+/).filter(w => w.length > 0).length`), which handles multiple spaces and line breaks correctly.

Multi-turn conversation structure: The request uses a `messages` array (OpenAI/Groq chat format) beginning with a `role: system` instruction. Each retry appends the model's previous response as a `role: assistant` turn, then adds the corrective `role: user` turn. The model sees the full conversation history, which prevents it from ignoring the constraint by treating it as a fresh instruction. This is the same technique used in human feedback fine-tuning loops.

Strict enforcement — no out-of-range copy ever proceeds: The brief requires the description to *strictly* be 90–150 words. After 5 attempts, if the copy is still out of range, the Code node `throws` an error. This stops the execution at that node, so a non-compliant post can never reach the human approval step or the publish nodes. In practice Groq's Llama 3.3 70B lands in range within the first one or two attempts (and responds in well under a second); the hard stop is the guarantee that nothing out of spec slips through. The returned object includes `wordCount` and `withinRange` for transparency in the execution log.

2. Image compositing pipeline for pixel-perfect brand consistency

The problem: n8n has no image manipulation primitive. AI image generation is unpredictable — the logo position and text placement vary every run. The brief requires a fixed structural template.

Approach: Python + Pillow, all layout values as named constants.

File: `backend/compositing.py`

```

IMAGE_SIZE = (1080, 1080) # canvas
LOGO_SIZE = (120, 120) # logo bounding box
LOGO_POS = (930, 30) # top-right corner, 30 px margin
TITLE_Y = 740 # top edge of title text block (lower third)
TITLE_X_CENTER = 540 # horizontal centre of canvas
TITLE_MAX_W = 900 # maximum line width in pixels
TITLE_MAX_LINES = 2 # hard cap on title lines
TITLE_FONT_SIZE = 48
TITLE_LINE_H = 68 # vertical gap between wrapped lines

```

Pipeline per run:

1. Open `assets/template.png` (1080×1080 navy brand card with gold separator).
2. Open `assets/logo.png`, resize to `LOGO_SIZE` with LANCZOS resampling, paste at `LOGO_POS`.
3. Wrap the topic title at `TITLE_MAX_W` pixels using `ImageFont.getlength()` for each candidate line — this is pixel-width measurement, not character count.
4. Truncate to `TITLE_MAX_LINES` lines; if truncated, replace the last line's end with "...".
5. Centre the text block: for each line, `x = TITLE_X_CENTER - line_width / 2`.
6. Draw each line at `y = TITLE_Y + line_index * TITLE_LINE_H`.
7. Return PNG bytes from an in-memory `io.BytesIO` buffer.

Guarantees: Logo is always at pixel (930, 30). Title always starts at y=740. The template image (background, separator, brand colour) never changes. The only dynamic input is the topic string. No matter how many topics run through the workflow, every card is structurally identical.

Asset generation: If `template.png` or `logo.png` are missing, `compositing.py` generates deterministic placeholders — a navy canvas with a gold separator for the template, a gold circle with "MH" initials for the logo. This means the backend works immediately after `pip install` with no manual asset setup required.

3. HITL pause and webhook response architecture

The problem: n8n is a stateless workflow engine. To pause execution and wait for a human to respond asynchronously (seconds to hours later), a mechanism is needed that:

- Does not poll (wasted CPU, rate limits)
- Survives a network interruption between n8n Cloud and the local FastAPI server
- Routes the human's decision back to the exact paused execution, not to a generic endpoint

What was built: n8n Wait node + FastAPI callback pattern

```

n8n execution reaches Wait node
└─ n8n serialises the execution state to disk / DB
└─ n8n generates a unique one-time webhook URL: $execution.resumeUrl
└─ execution is SUSPENDED — no CPU used, no polling

```

```

FastAPI receives $execution.resumeUrl in POST /request-approval

```

```

└ stores it in _reviews dict keyed by review_id
└ returns { review_id, review_url } to n8n (before suspension)

Human opens /review/{review_id} in a browser
└ sees composited image, generated copy, textarea for editing
└ clicks Approve / Edit & Approve / Reject

POST /review/{review_id}/decision fires
└ FastAPI calls: requests.post(record["resume_webhook_url"], json={
    "decision": "approved" | "rejected",
    "final_copy": edited_copy or original_copy,
    "review_id": review_id
})
└ n8n receives the POST, RESUMES the execution
└ the POST body becomes $json for the next node (Was Approved? IF node)

```

Key design decisions:

- **\$execution.resumeUrl is passed before the Wait node executes.** The sequence is: Request Human Approval (HTTP Request) → Wait for Human Decision. When n8n executes the HTTP Request node, the execution is still running, so \$execution.resumeUrl is available. The Wait node suspends *after* FastAPI has already stored the URL.
- **Decision payload includes final_copy, not just decision.** This allows the Edit & Approve path to pass through the human's rewritten copy without a separate storage lookup. final_copy flows directly into the mock publish nodes as \$json.final_copy.
- **In-memory store (_reviews dict) is intentional for this demo.** A production system would use Redis or a database. The store holds { copy, image_url, resume_webhook_url, status } per review. On rejection or approval, status is updated to prevent double-firing.
- **callback_status is returned in the decision response.** The FastAPI /review/{id}/decision endpoint returns the HTTP status code it received when it called the n8n resume webhook. This surfaces webhook delivery failures without crashing the human-facing response.